

Data-Driven Horror: Analyzing Horror Movies with Python

CS 2435 - Introduction to Scientific Programming
Presented by Christian Etherson



Project Overview

- Collected horror movie data using APIs and web scraping
- Cleaned, parsed, and integrated data from multiple sources
- Explored relationships between movie characteristics and commercial performance
- Communicated findings through statistical visualizations

Building the Dataset

- Initially planned to use the OMDb API as the primary data source
- Identified a limitation: the API does not support retrieving all horror films by genre
- Developed a Python web scraper to collect horror movie titles from an online list
- Used the scraped titles as input for subsequent API requests

Web Scraping Implementation

```
import requests
import csv
from bs4 import BeautifulSoup

#Program to collect horror movie titles

#Web scraper code that scrapes movie titles from a website list
def scrapeHorrorMovies():
    page = requests.get("https://horror.fandom.com/wiki/Chronological_List_of_Horror_Films")
    soup = BeautifulSoup(page.content, "html.parser")
    list_items = soup.find_all("li")
    items = [item.text.strip() for item in list_items]
    titles = ", ".join(items).split(", ")
    return titles
```

Python script used to scrape horror movie titles, creating the input dataset for API requests.

Retrieving Movie Data with the OMDb API

- Iterated through the scraped movie titles
- Queried the OMDb API for each film
- Retrieved structured metadata as JSON objects
- Stored each movie's data as a dictionary for further processing and analysis

API Data Collection

Python workflow for requesting movie metadata from the OMDb API and storing the results in a structured format.

```
def getHorrorMovieData():
    titles = scrapeHorrorMovies()
    horrorMovies = list()
    base_url = "http://www.omdbapi.com/"
    api_key = "953a5ba8"
    for title in titles:

        params = {
            "apikey": api_key,
            "t": title
        }
        response = requests.get(base_url, params = params)
        if response.status_code == 200:
            try:
                movie = response.json()

                if movie.get("Response") == "False":
                    continue

                elif movie.get("Type") == "series":
                    continue

                else:
                    print(movie)
                    horrorMovies.append(movie)
            except:
                continue
    return horrorMovies
```

Data Science Toolkit

Libraries Used

- Pandas: data cleaning and manipulation
- Numpy: numerical operations
- Matplotlib: data visualization
- SciPy: statistical analysis



Data Cleaning & Preparation

- Converted raw API responses into a structured Pandas DataFrame
- Removed TV series and non-horror titles
- Filtered to English-language films released between 1970-2024
- Handled missing values and excluded irrelevant variables
- Produced a clean master dataset for analysis

Data Cleaning Workflow

Code used to create and clean the analysis dataset.

```
#Create dataframe from list and view it
horrorDataDf = pd.DataFrame(horrorData)
print(horrorDataDf.to_string())
```

```
#Clean the dataframe by including only English movies from 1970-2024
horrorDataDfClean = horrorDataDf[horrorDataDf["Language"].apply(lambda x: "English" in x.split(', '))]
horrorDataDfClean = horrorDataDfClean[horrorDataDfClean["Genre"].apply(lambda x: "Horror" in x.split(', '))]
horrorDataDfClean = horrorDataDfClean.drop_duplicates(subset = ["imdbID"])
horrorDataDfClean["Year"] = pd.to_numeric(horrorDataDfClean["Year"])
horrorDataDfClean = horrorDataDfClean[(horrorDataDfClean["Year"] >= 1970) & (horrorDataDfClean["Year"] <= 2024)]
horrorDataDfClean.drop(columns = ["Production", "Website", "Response",
                                   "DVD", "Type", "Metascore", "Ratings",
                                   "Poster", "Awards", "Released", "Country",
                                   "Plot", "Actors", "Writer"], inplace = True)
print(horrorDataDfClean.sort_values(by = "Year").to_string())
```

Research Questions



Question 1: Audience Ratings & Financial Success

- Is IMDb rating associated with box office performance among horror films?

Question 2: Title Characteristics & Performance

- Do horror movies with one-word titles have stronger box office performance compared to movies with multi-word titles?

Analytical Approach

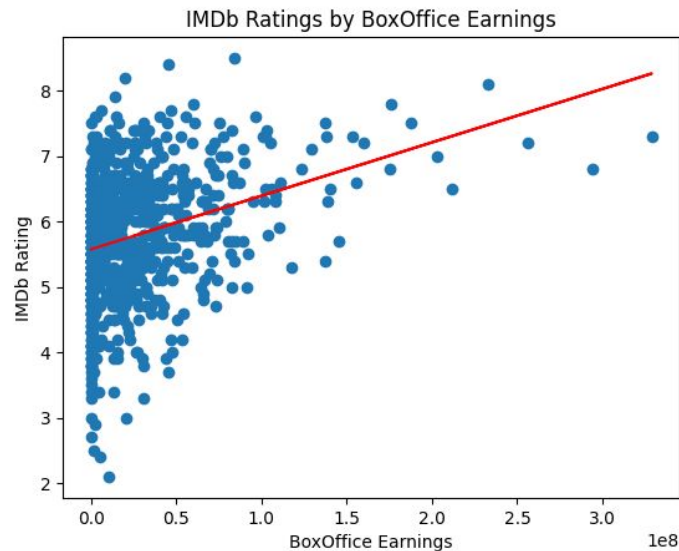
- Segmented the cleaned dataset into subsets tailored to each research question
- Visualized variable relationships using scatterplots with fitted trendlines
- Quantified relationships using Pearson correlation coefficients
- Applied two-sample hypothesis tests to assess statistical significance

Calculating and Visualizing the Rating - Revenue Relationship

```
#View the correlation coefficient and plot it on the graph
print(horrorDataLr.corr())
reg = np.polyfit(horrorDataLr["BoxOffice"], horrorDataLr["imdbRating"], deg = 1)
trend = np.polyval(reg, horrorDataLr["BoxOffice"])
plt.scatter(x = horrorDataLr["BoxOffice"], y = horrorDataLr["imdbRating"])
plt.plot(horrorDataLr["BoxOffice"], trend, "r")
plt.title("IMDb Ratings by BoxOffice Earnings")
plt.xlabel("BoxOffice Earnings")
plt.ylabel("IMDb Rating")
```

Finding 1: Rating vs. Box Office (All Films)

- Correlation coefficient: $r = 0.31$: a weak-to-moderate positive relationship
- Higher-grossing horror films tend to have slightly higher IMDb ratings, but the relationship is noisy at lower box office levels



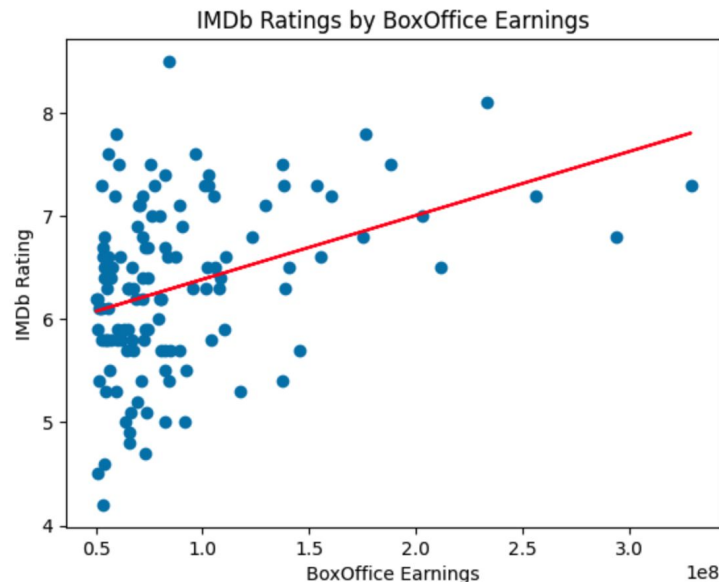
	imdbRating	BoxOffice
imdbRating	1.000000	0.309232
BoxOffice	0.309232	1.000000

Refining the Analysis: Isolating High-Grossing Films

```
#Examine the same correlation but only for movies with a box office earning of at least 50 million dollars.
horrorDataLrOneMil = horrorDataLr[horrorDataLr["BoxOffice"] >= 50000000]
print(horrorDataLrOneMil.to_string())
print(horrorDataLrOneMil.corr())
reg = np.polyfit(horrorDataLrOneMil["BoxOffice"], horrorDataLrOneMil["imdbRating"], deg = 1)
trend = np.polyval(reg, horrorDataLrOneMil["BoxOffice"])
plt.scatter(x = horrorDataLrOneMil["BoxOffice"], y = horrorDataLrOneMil["imdbRating"])
plt.plot(horrorDataLrOneMil["BoxOffice"], trend, "r")
plt.title("IMDb Ratings by BoxOffice Earnings")
plt.xlabel("BoxOffice Earnings")
plt.ylabel("IMDb Rating")
```

Finding 2: Rating vs. Box Office (Films Grossing $\geq \$50M$)

- Correlation coefficient increases to $r = 0.37$ among top-grossing films
- Suggests the rating/revenue relationship strengthens once a film reaches wider commercial success



	imdbRating	BoxOffice
imdbRating	1.00000	0.36754
BoxOffice	0.36754	1.00000

Preparing the Data: Comparing Title Length

```
#Create a dataframe only including horror titles with one word.
horrorDataOneWord = horrorDataDfClean[horrorDataDfClean["Title"].str.split().str.len() == 1]
horrorDataOneWord.drop(columns = ["Year", "Rated", "Runtime",
                                   "Genre", "Director", "Language",
                                   "imdbVotes", "imdbID"], inplace = True)
horrorDataOneWord = horrorDataOneWord[horrorDataOneWord["BoxOffice"] != "N/A"]
horrorDataOneWord = horrorDataOneWord[horrorDataOneWord["imdbRating"] != "N/A"]
horrorDataOneWord['BoxOffice'] = horrorDataOneWord['BoxOffice'].str.replace('$', '').str.replace(',', '')
horrorDataOneWord["imdbRating"] = pd.to_numeric(horrorDataOneWord["imdbRating"])
horrorDataOneWord["BoxOffice"] = pd.to_numeric(horrorDataOneWord["BoxOffice"])
print(horrorDataOneWord.to_string())

#Create a data frame with horror titles with multiple words.
horrorDataMulti = horrorDataDfClean[horrorDataDfClean["Title"].str.split().str.len() != 1]
horrorDataMulti.drop(columns = ["Year", "Rated", "Runtime",
                                "Genre", "Director", "Language",
                                "imdbVotes", "imdbID"], inplace = True)
horrorDataMulti = horrorDataMulti[horrorDataMulti["BoxOffice"] != "N/A"]
horrorDataMulti = horrorDataMulti[horrorDataMulti["imdbRating"] != "N/A"]
horrorDataMulti['BoxOffice'] = horrorDataMulti['BoxOffice'].str.replace('$', '').str.replace(',', '')
horrorDataMulti["imdbRating"] = pd.to_numeric(horrorDataMulti["imdbRating"])
horrorDataMulti["BoxOffice"] = pd.to_numeric(horrorDataMulti["BoxOffice"])
print(horrorDataMulti.to_string())
```


Summary Statistics: One-Word vs. Multi-Word Titles

One-word titles: mean IMDb rating 5.90, mean box office ~\$21.6M (n = 224)

Multi-word titles: mean IMDb rating 5.73, mean box office ~\$25.1M (n = 627)

```
horrorDataOneWord.describe()
```

	imdbRating	BoxOffice
count	224.000000	2.240000e+02
mean	5.899107	2.155612e+07
std	0.873680	3.603018e+07
min	3.400000	2.520000e+02
25%	5.400000	5.634765e+05
50%	5.900000	7.551136e+06
75%	6.500000	2.805996e+07
max	8.500000	3.288750e+08

```
horrorDataMulti.describe()
```

	imdbRating	BoxOffice
count	627.000000	6.270000e+02
mean	5.727592	2.505590e+07
std	0.959793	3.542810e+07
min	2.100000	9.500000e+01
25%	5.150000	1.054926e+06
50%	5.800000	1.329442e+07
75%	6.400000	3.500827e+07
max	8.400000	2.940728e+08

Setting Up the Hypothesis Tests

```
#Create a function that classifies is a movie has one word or multiple words in the title.
def oneOrMore(row):
    if len(row["Title"].split()) == 1:
        return "OneWord"
    else:
        return "MultiWord"

#Create a merged dataframe from the two of them and conduct a hypothesis test
horrorDataTitle = horrorDataDfClean.drop(columns = ["Year", "Rated", "Runtime",
                                                    "Genre", "Director", "Language",
                                                    "imdbVotes", "imdbID"])
horrorDataTitle = horrorDataTitle[horrorDataTitle["BoxOffice"] != "N/A"]
horrorDataTitle = horrorDataTitle[horrorDataTitle["imdbRating"] != "N/A"]
horrorDataTitle['BoxOffice'] = horrorDataTitle['BoxOffice'].str.replace('$', '').str.replace(',', '')
horrorDataTitle["imdbRating"] = pd.to_numeric(horrorDataTitle["imdbRating"])
horrorDataTitle["BoxOffice"] = pd.to_numeric(horrorDataTitle["BoxOffice"])
horrorDataTitle["Words"] = horrorDataTitle.apply(oneOrMore, axis = 1)
print(horrorDataTitle.to_string())
```

Hypothesis Test: Box Office Earnings

- H_0 : No difference in box office earnings between one-word and multi-word titles
- H_a : A difference exists
- Result: $p = 0.207$, fail to reject H_0 ; no statistically significant difference in box office earnings by title length

```
#Use the dataframe to conduct a hypothesis test
#H0: There is no difference in box office earnings between horror movie titles with one word vs. multiple words
#HA: There is a difference in box office earnings between horror movie titles with one word vs. multiple words
multipleWords = horrorDataTitle[horrorDataTitle["Words"] == "MultiWord"]["BoxOffice"]
oneWord = horrorDataTitle[horrorDataTitle["Words"] == "OneWord"]["BoxOffice"]

t_value, p_value = stats.ttest_ind(multipleWords, oneWord)

print(p_value)
```

0.20679416648543147

Hypothesis Test: IMDb Ratings

- H_0 : No difference in IMDb ratings between one-word and multi-word titles
- H_a : A difference exists
- Result: $p = 0.019$, reject H_0 ; there is a statistically significant difference in ratings by title length

```
#Use the dataframe to conduct a hypothesis test
#H0: There is no difference in IMDb ratings between horror movie titles with one word vs. multiple words
#HA: There is a difference in IMDb ratings between horror movie titles with one word vs. multiple words
multipleWords = horrorDataTitle[horrorDataTitle["Words"] == "MultiWord"]["imdbRating"]
oneWord = horrorDataTitle[horrorDataTitle["Words"] == "OneWord"]["imdbRating"]

t_stat, p_value = stats.ttest_ind(multipleWords, oneWord)
print(p_value)
```

0.019041827894378707

Conclusion & Takeaways

Complete pipeline built:

web scraping → API integration → data cleaning → statistical analysis

- Found a moderate positive correlation between IMDb rating and box office earnings, strengthening among top-grossing films
- Title length showed no significant effect on box office performance, but a significant effect on IMDb ratings
- Translated prior data science experience from R into Python, gaining hands-on practice with Pandas, NumPy, SciPy, and Matplotlib

Thank You!
